arXiv:2406.13542v3 [cs.CL] 18 Jul 2024

SELF-PLAY WITH EXECUTION FEEDBACK: IMPROVING INSTRUCTION-FOLLOWING CAPABILITIES OF LARGE LANGUAGE MODELS

Guanting Dong*, **Keming Lu**, **Chengpeng Li***, **Tingyu Xia***, **Bowen Yu**†
Chang Zhou, **Jingren Zhou**
Qwen Team, Alibaba Inc.
{dongguanting.dgt,lukeming.lkm,lichengpeng.lcp}@alibaba-inc.com
{xiatingyu.xty, yubowen.ybw,ericzhou.zc,jingren.zhou}@alibaba-inc.com

ABSTRACT

One core capability of large language models (LLMs) is to follow natural language instructions. However, the issue of automatically constructing high-quality training data to enhance the complex instruction-following abilities of LLMs without manual annotation remains unresolved. In this paper, we introduce AUTOIF, the first scalable and reliable method for automatically generating instruction-following training data. AUTOIF transforms the validation of instruction-following data quality into code verification, requiring LLMs to generate instructions, the corresponding code to check the correctness of the instruction responses, and unit test samples to verify the code's correctness. Then, execution feedback-based rejection sampling can generate data for Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) training. AUTOIF achieves significant improvements across three training algorithms, SFT, Offline DPO, and Online DPO, when applied to the top open-source LLMs, Qwen2 and LLaMA3, in self-alignment and strong-to-weak distillation settings. Our code is publicly available at https://github.com/QwenLM/AutoIF.

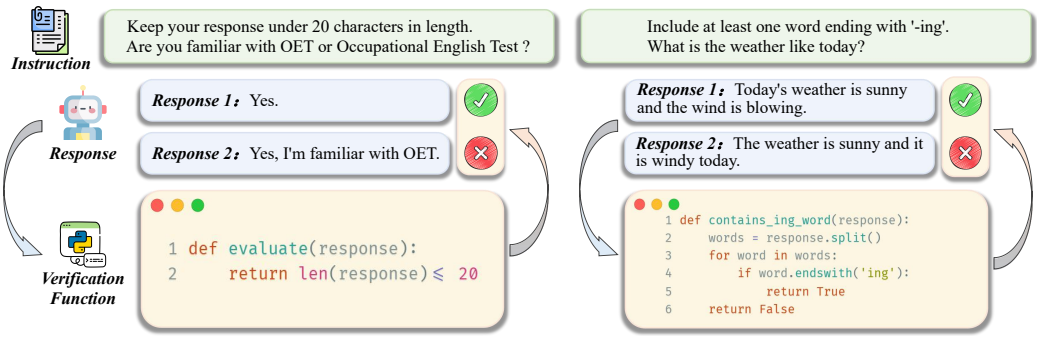

Figure 1: An example of the verification function automatically assesses the adherence of responses to the instruction's constraints.

1 INTRODUCTION

The instruction-following ability of large language models (LLMs) refers to their capacity to understand, interpret, and execute commands given to them in natural language (Lou et al., 2023; OpenAI et al., 2024). This ability is fundamental to contemporary LLMs as it enables them to leverage their underlying knowledge, interact intuitively with users (Ouyang et al., 2022), adapt to various requirements (Zhang et al., 2023), and perform complex tasks (Sun et al., 2024). Misunderstandings in

*Work done during internship at Qwen team, Alibaba Inc.
†Corresponding author

following instructions can lead to unintended outcomes, potentially resulting in severe consequences, particularly in critical scenarios (Zhou et al., 2023; Chang et al., 2024).

Although instruction following is crucial, scalable and reliable methods to enhance this capability of LLMs remain elusive. Current efforts in this field are divided into manual annotation (Wei et al., 2021; Zhou et al., 2023; Jiang et al., 2024b) and behavior imitation (Xu et al., 2023; Zhao et al., 2024). Manual annotation involves annotators designing instructions and writing corresponding responses. However, due to human cognition's limitations, creating highly complex and diverse instructions is challenging, making the process difficult to scale. Furthermore, accurately executing complex instructions can sometimes be difficult for humans (Sun et al., 2024; Cao et al., 2024b), requiring multiple rounds of rigorous and costly validation (Wang et al., 2024a; Wei et al., 2024). On the other hand, behavior imitation aims to distill responses from more advanced LLMs (Taori et al., 2023; Peng et al., 2023) like GPT-4. This approach limits models to the capabilities of the advanced LLMs from which they are distilled. Moreover, even advanced LLMs can make mistakes, and the reliability of the distilled data cannot be guaranteed (Cui et al., 2023). Consequently, models trained with this data may have a propensity to not follow instructions accurately (Zhou et al., 2024).

In this paper, we introduce AUTOIF, the first scalable and reliable method for automatically generating instruction following training Data for Supervised Finetuning (SFT) or Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022). The core idea of AUTOIF is to use code to verify the correctness of following instructions. Intuitively, if designed properly, a significant portion of instructions, such as "Keep your response under 20 characters in length" can be verified for correctness using code, as illustrated in Fig. 1. Therefore, the key components of AUTOIF include (1) automatically generating instructions that can be verified by code, (2) automatically generating corresponding verification codes for these instructions, and (3) ensuring the reliability of the first two steps. Specifically, we start by providing AUTOIF with a small set of hand-written seed instructions. Then, LLMs, not necessarily advanced ones, generate an augmented instruction set through self-instruct (Wang et al., 2023). Next, LLMs write verification codes and unit test cases for each instruction. Only the code that compiles correctly, passes the test cases, and back-translates to the original instruction is retained. If an instruction does not have a corresponding code that can verify its correctness, it is discarded. Finally, we employ LLMs to generate responses that either pass or fail the verification code using execution feedback-based rejection sampling (Yuan et al., 2023). Responses that pass can be directly used for SFT, while pairs of passing and failing responses can be used to create chosen-rejected pairs for Direct Preference Optimization (DPO) (Rafailov et al., 2023) and other RLHF algorithms. Moreover, once the instructions and verification code are determined, this process can be conducted on-policy, continually enhancing the instruction-following capabilities.

Through extensive experiments, we have demonstrated that AUTOIF achieves significant improvements under three training algorithms—SFT, Offline DPO, and Online DPO—when applied to the top open-source LLMs, Qwen2-72B and LLaMA3-70B, in both self-alignment and strong-to-weak distillation settings. In the IFEval benchmark, we achieved Loose Instruction (Acc.) rates of up to 88.0% with Qwen2-72B and 90.4% with LLaMA3-70B, marking the first instance of surpassing 90% accuracy. In the FollowBench benchmark, these models also showed significant improvements, with increases of over 5% in the SSR metric (avg). Additionally, they enabled Qwen2-7B and LLaMA3-8B to achieve average performance gains of over 4% in both benchmarks. Replacing Qwen2-72B and LLaMA3-70B with the more advanced GPT-4 resulted in further substantial improvements. We will open-source the SFT and DPO datasets constructed using AUTOIF on Qwen2-72B, representing the first open-source complex instruction-following dataset at a scale of tens of thousands.

2 RELATED WORKS

Instruction-following capabilities are among the most essential features of LLMs (OpenAI et al., 2024; Lou et al., 2023), which are expected to precisely follow a broad and complex set of instructions. Consequently, recent research has concentrated on evaluating LLMs' instruction-following abilities in various contexts, such as verifiable (Zhou et al., 2023), compositional (Qin et al., 2024), format-related (Xia et al., 2024), refuting (Yan et al., 2024), and fine-grained instructions (Jiang et al., 2024b). However, a significant gap remains between open-source and proprietary closed-source LLMs. Sun et al. (2024) propose Conifer, which enhances the instruction-following capabilities of open-source LLMs through knowledge distillation from proprietary LLMs. Wang et al. (2024b)

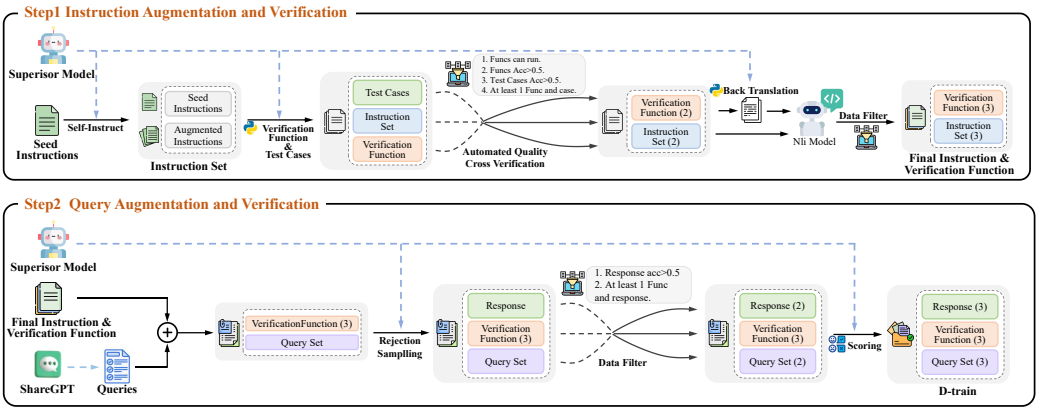

Figure 2: An Overview of AUTOIF: A Two-Stage Automated Instruction-Following Data Synthesis Method.

use LLMs to encode instruction metadata and augment diverse instructions from this metadata, employing proprietary LLMs for quality control. Both approaches, however, rely on proprietary LLMs for response distillation or judgment, which not only limits their potential but also subjects them to OpenAI's terms of use [1]. In this work, we propose AUTOIF, a more scalable and reliable method to enhance the instruction-following capabilities of LLMs. AUTOIF uses execution feedback from self-generated verification functions to provide supervision for instructions. This allows for effective self-alignment and strong-to-weak distillation on open-source models, thereby narrowing the performance gap with proprietary LLMs.

Learning with Execution Feedback is a widely-used technique in automated alignment for tool use and coding (Cao et al., 2024a). These learning methods typically utilize execution feedback from tools such as code executors to provide supervision for specific tasks. For instance, Le et al. (2022) employ feedback from unit tests via code compilers to enhance code synthesis capabilities through reinforcement learning. Similarly, Chen et al. (2023) train LLMs to provide debugging suggestions as feedback to improve coding abilities. Additionally, Qiao et al. (2024) introduce Reinforcement Learning with execution feedback to enhance LLMs using execution results from tools. Building on this learning paradigm, we propose a novel scalable oversight method that enables LLMs to autonomously generate verification functions and unit tests for natural language instructions, thereby applying execution feedback to enhance their instruction-following capabilities.

3 AUTOIF

We introduce AUTOIF, an automated, scalable, and reliable method designed to enhance the instruction-following capabilities of LLMs. In this section, we outline the preliminaries (§3.1), detail the two core components of AUTOIF (§3.2, §3.3), and discuss various training strategies that can be seamlessly integrated with AUTOIF (§3.4).

3.1 PRELIMINARIES

Instruction-following Capabilities. Following instructions is one of the most crucial skills in modern LLMs. These models are expected to provide precise responses to queries containing complex instructions, which can be either atomic or compositional. To evaluate the instruction-following capability of LLMs, we define a general instruction-following requirement as a specific task. In this task, given an instruction $I = \{i_j\}_{j=1}^{N}$ with N specific constraints (e.g. "Please generate text in Shakespearean style, no more than 50 tokens" contains 2 constraints) and a specific query x, an LLM π_θ should generate precise response $y \sim \pi_\theta(y \mid x, I)$ adhering to the constraints.

[1] https://openai.com/policies/terms-of-use

Verifiable Instructions. The complexity and diversity of instructions necessitate manual construction and verification for reliable supervision. This practical challenge motivates us to focus initially on instructions that can be automatically verified through programs and code executors, also known as verifiable instructions (Zhou et al., 2023). Specifically, for a given instruction I and task-specific query q, there exists a verification function f_I such that $f_I(y)$ returns true when the model's response y correctly follows the instruction. We demonstrate that supervision of such instructions can be self-generated through scalable oversight with LLMs and execution feedback. Extensive experiments in our work show that training on verifiable instructions significantly benefits the handling of other general instructions that are more complex but unverifiable with simple code snippets.

Method Overview. AUTOIF synthesizes high-quality instruction-following data through self-evolution, rejection sampling, and execution feedback. As illustrated in Fig. 2, AUTOIF integrates automated data augmentation with quality verification processes, including automatically generated verification functions and back-translation instructions. This approach enables a two-stage automated data synthesis at both the instruction (§3.2) and query levels (§3.3). Additionally, we introduce three training strategies (§3.4) and explore two experimental settings (§4.1) to thoroughly evaluate the effectiveness and generalization of AUTOIF.

3.2 INSTRUCTION AUGMENTATION AND VERIFICATION

We first develop verifiable instructions along with corresponding evaluation functions, using rejection sampling informed by execution feedback.

Seed Instruction Construction. We start by handwriting a set of seed instructions, denoted as D_{seed}, ensuring that each instruction contains only a single atomic constraint (e.g., "Answer the words that begin with B"). Detailed information on seed instructions is listed in Appx. §A.

Self-Instruct. Self-Instruct (Wang et al., 2023) is a straightforward and intuitive strategy for automated data augmentation that has garnered significant attention in the field of LLM reasoning (Xu et al., 2023; Zhao et al., 2023). For each instruction in D_{seed}, we use an LLM to perform K instruction rewrites, generating D_{aug}. We then combine the seed and augmented data sets to obtain an enhanced set of instructions, $D_{ins} = D_{seed} \cup D_{aug}$, and remove any duplicates.

Automated Quality Cross Verification. Previous research has shown that relying solely on model-generated augmented instructions often leads to the inclusion of low-quality samples (Mumuni & Mumuni, 2022; Xie et al., 2020; Zheng et al., 2024). Inspired by a series of tool execution studies, we employ an LLM to generate verification functions and test cases for each instruction. We use feedback from executing Python programs to ensure quality control. Given the instruction set D_{ins}, the LLM M employs a rejection sampling (Touvron et al., 2023; Yuan et al., 2023) to generate K verification functions $f_I = \{f_i\}_{j=1}^{K}$ and test cases $c_I = \{c_i\}_{j=1}^{K}$ for each instruction I, resulting in the set $\{I, f_I, c_I\} \in D_{ins}$. We then cross-validate the quality of the instructions using the verification functions and test cases, ensuring they meet the following criteria:

- The verification function $f \in f_I$ can be successfully compiled by the Python executor.
- Each test case $c \in c_I$ achieves an accuracy rate greater than 0.5 across all verification functions.
- Each verification function $f \in f_I$ achieves an accuracy rate greater than 0.5 across all test cases.
- Each instruction includes at least one evaluation function and test case.

By adhering to these four conditions, we obtain the quality-filtered instruction set $\{I^{(2)}, f_I^{(2)}\} \in D_{ins}^{(2)}$.

Back-translation Verification. After the cross-validation stage, we obtained initially quality-verified verification functions and instructions. To further ensure the consistency between instructions and verification functions, we introduce back-translation. For a given pair $\{I^{(2)}, f_I^{(2)}\} \in D_{ins}^{(2)}$, we use the LLM M to back-translate the verification function $f \in f_I^{(2)}$ into instruction I_f. We then treat I as the *premise* and the back-translated instruction I_f as the *hypothesis*. Using the NLI model, we identify the semantic relationship between the two instructions. The prediction can fall into one of three categories: *entailment*, *contradiction*, or *neutral*:

$$p_\theta(\cdot \mid q, q_{\text{aug}}) = \text{softmax}\left(\text{score}_\theta(I, I_f)\right), \tag{1}$$

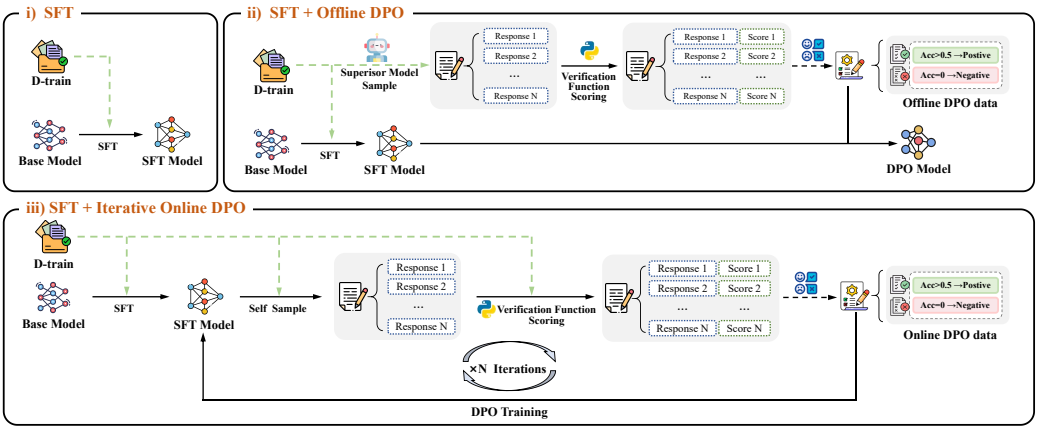

Figure 3: Different training strategies that can be adapted with synthetic dataset generated by AUTOIF.

where $\text{score}_\theta : \mathbb{R}^{k\times\ell_I} \times \mathbb{R}^{k\times\ell_{I_f}} \rightarrow \mathbb{R}^3$ is a model dependent scoring function with parameters θ. We filter out any instruction I labeled as *contradiction* to ensure the intent consistency. Finally we obtain the set $\{I^{(3)}, f_I^{(3)}\} \in D_{ins}^{(3)}$

3.3 QUERY AUGMENTATION AND VERIFICATION

Once we have obtained verified instructions and verification functions, we utilize them to create training data comprising queries and responses.

Query Reforming and Augmentation. In the real-world application of modern chatbots, instructions are typically employed to generate constrained responses to user queries. Therefore, creating high-quality instructions is merely the initial step toward achieving effective instruction-following capabilities. To acquire authentic queries, as shown in the bottom part of Fig. 2, we randomly selected K user queries from ShareGPT (Chiang et al., 2023) for each instruction and concatenated them to construct the seed query dataset $x, f_I^{(3)} \in D_q$. To further enhance the diversity and complexity of the input x, we utilized the LLM to generate K responses $y_x = \{y_i\}_{i=1}^K$, resulting in $\{x, f_I^3, y_x\} \in D_q$.

Instruction-following Verification. Following the previous quality cross-verification process, we further employ verification functions to assess whether the augmented responses adhere to the constraints in input x. Similarly, we require each response in D_q to meet the following conditions:

- Each response must achieve an accuracy rate greater than 0.5 across all verification functions.
- Each input must include at least one verification function and one response.

Based on these rules, we obtain the set $(x^{(2)}, f_I^{(3)}, y^{(2)}) \in D_q^{(2)}$.

Query Quality Verification. Additionally, we observe that concatenated instructions and queries often conflict. For instance, a high-quality response to the query "help me write a news article" is unlikely to comply with the instruction "please limit your answer to two words". Such high-level semantic inconsistencies are challenging for a simple NLI model to discern. Therefore, we employ the LLM M to assign matching scores between the instruction and query in input $x^{(2)}$ and the corresponding responses $y^{(2)}$, on a scale from 1 to 10. We then filter out samples with a score lower than 8, constructing the final training set $D_{\text{train}} = \{x_i, y_i, f_{Ii}\}_{i=1}^N$.

3.4 TRAINING STRATEGIES

AUTOIF offers multifaceted supervision for the instruction-following task, making it adaptable to various training strategies. To thoroughly evaluate the effectiveness of AUTOIF, we propose the following training approaches:

Supervised Fine-tuning (SFT). Given $(x_i, y_i) \in D_{\text{final}}$, we apply the standard Supervised Fine-tuning (SFT) objective on the base model P with parameters θ: $\mathcal{L}(\boldsymbol{\theta}) = \sum_{(x_i,y_i)\in\mathcal{D}_{\text{train}}} \log \mathbb{P}_\theta(y_i \mid x_i)$, where x_i denotes the i-th input, consisting of a concatenated instruction and user query.

SFT + Offline DPO. In the process of AUTOIF, multiple scales of quality filtering are utilized, naturally generating a substantial number of positive and negative sample pairs. This motivates us to obtain pairwise preference data (x, y_w, y_l). Our preference data mining is divided into two parts:

- **Instruction Level:** During the automated quality cross-verification stage, we first extract positive samples c_w from cases with an accuracy rate higher than 0.5 on all verification functions and negative samples c_l from cases with an accuracy rate of 0. We then construct pairwise preference data for each instruction: $D^{\text{pref}}_{\text{ins}} \rightarrow (I, c_w, c_l)$.
- **Query Level**: In the query quality verification process, we similarly extract positive samples y_w from responses with an accuracy rate higher than 0.5 on all verification functions and negative samples y_l from responses with an accuracy rate of 0. We then construct query preference data: $D^{\text{pref}}_{\text{query}} \rightarrow (x, y_w, y_l)$.

Finally, we merge the two parts of the data: $D_{\text{pref}} = D^{\text{pref}}_{\text{ins}} \cup D^{\text{pref}}_{\text{query}}$. To further explore the potential of pairwise preference data $(x, y_w, y_l) \in D_{\text{pref}}$, we first perform vanilla SFT on the base model π_θ to obtain an SFT model π^{SFT}_θ as equation 3.4. Then, we apply Direct Preference Optimization (DPO) (Rafailov et al., 2024) on our SFT model, which can be formulated as follows:

$$\mathcal{L}_{\text{DPO}}(\pi^{\text{SFT}}_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log\sigma(\beta\log\frac{\pi^{\text{SFT}}_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi^{\text{SFT}}_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})], \tag{2}$$

where the reference model π_{ref} is set to π^{SFT}_θ initially and remains fixed throughout training. β is a hyperparameter and σ is the sigmoid function. $\mathcal{L}_{\text{DPO}}$ aims to maximize the log probability of preferred y_w relative to the dispreferred y_l.

SFT + Iterative Online DPO. Online training enables real-time, iterative optimization of model weaknesses. It relies on high-quality, lightweight reward models to provide continuous supervision feedback. In the case of AUTOIF, verification functions serve as rigorous filtering standards, akin to reward models, delivering immediate feedback on model responses across training iterations. Following offline DPO, we conduct initial SFT on the base model π_θ to derive an SFT model π^{SFT}_θ with initial instruction-following capabilities. As depicted in Fig. 3, we set the generation temperature to 0.8 and allow the SFT model to generate K responses through self-sampling for each training sample, forming a response set $\{R_1, \ldots, R_k\}$. Then, we employ corresponding verification functions to assess K responses, thereby constructing the online DPO dataset $D^{\text{pref}}_{\text{online}} = (x, y_w, y_l)$ based on average pass rates across all functions. Finally, leveraging D_{online}, we sequentially perform DPO training on π^{SFT}_θ. Importantly, our iterative online optimization process progressively unlocks enhanced instruction-following capabilities.

4 EXPERIMENT

4.1 EXPERIMENTAL SETUP

Datasets & Baselines. We conduct experiments using two LLMs from the Qwen2 series (Qwen2-7B and Qwen2-72B-Instruct) and two from the LLaMA3 series (LLaMA3-8B and LLaMA3-70B-Instruct). Please note that the AUTOIF method proposed in this work has been employed in the open-source Qwen2-Instruct model. Thus, the version of Qwen2-Instruct we utilized represents an early iteration during internal development, rather than the final open-source model. The training datasets are respectively generated from Qwen2-72B-Instruct and LLaMA3-70B-Instruct, with detailed statistics provided in Tab. 5. We demonstrate the effectiveness of AUTOIF by evaluating the instruction-following capabilities of models fine-tuned with self-generated datasets using AUTOIF. Additionally, we include strong open and closed-source LLM baselines such as Mixtral-8x22B and GPT-4. For more details, refer to Appx. §B.

Settings. In our experiments, we mainly explore two experimental setups:

Table 1: The main results on two instruction-following and four general benchmarks. Pr. and Ins. stand for prompt and instruction levels, respectively. S and L represent strict and loose metrics for IFEval. The subscript indicates the increase in metrics compared to the corresponding backbone model. The highest accuracy for each setup is highlighted in green. Results marked with † are directly sourced from the original benchmarks.

Model	IFEval				FollowBench (SSR)						C-Eval	MMLU	GSM8k	HumanEval
	Pr (S)	Pr. (L)	Ins. (S)	Ins. (L)	Level 1	Level 2	Level 3	Level 4	Level 5	Avg				
Baselines (< 10B)														
Qwen2-7B	37.7	43.6	49.4	53.4	55.6	53.5	53.7	49.9	48.6	52.3	74.4	64.4	71.1	58.1
Qwen2-7B(ShareGPT)	30.9	33.5	42.4	45.2	56.1	52.7	50.8	45.2	47.9	50.5	70.2	59.8	59.4	52.4
LLaMA3-8B	24.6	26.1	38.1	39.7	10.0	10.3	10.5	14.3	12.7	11.6	24.2	38.8	4.5	0.6
LLaMA3-8B(ShareGPT)	23.7	26.4	33.8	37.1	44.0	40.0	39.6	33.3	33.6	38.1	35.2	44.6	20.5	38.1
Mistral-7B	23.3	24.6	38.4	39.6	40.1	39.7	37.9	35.7	36.7	38.0	38.2	47.6	20.5	38.4
Baselines (> 10B)														
Qwen2-72B-Instruct	77.1	80.4	84.4	86.9	70.2	66.6	63.5	58.1	56.3	62.9	83.8	80.8	87.9	73.8
LLaMA3-70B-Instruct	77.8	83.8	84.2	88.8	60.7	60.5	61.1	61.7	60.3	60.9	60.2	80.5	92.6	78.7
Mixtral-8x22B	41.8	47.3	55.2	60.0	63.9	60.0	58.2	56.2	55.3	58.7	-	-	-	-
GPT-4†	76.9	79.3	83.6	85.4	84.7	77.6	76.2	77.9	73.3	77.9	-	-	-	-
GPT-3.5 Turbo†	-	-	-	-	80.3	71.2	74.2	69.6	67.1	72.5	-	-	-	-
Supervision Model: Qwen2-72B														
Strong-to-Weak														
Qwen2-7B-SFT	$40.7_{+3.0}$	$44.5_{+0.9}$	$51.3_{+1.9}$	$55.4_{+2.0}$	$60.2_{+4.6}$	$53.7_{+0.2}$	$54.3_{+0.6}$	$49.9_{+0.0}$	$48.6_{+0.0}$	$53.3_{+1.0}$	$73.9_{+0.0}$	$64.4_{+0.0}$	$74.1_{+3.0}$	$58.3_{+0.2}$
w/ Offline DPO	$41.2_{+3.5}$	$44.7_{+1.2}$	$51.4_{+2.0}$	$56.2_{+2.8}$	$61.4_{+5.8}$	$54.5_{+1.0}$	$54.3_{+0.6}$	$51.2_{+1.3}$	$48.6_{+0.0}$	$54.0_{+1.7}$	$75.1_{+0.7}$	$64.5_{+0.1}$	$72.9_{+1.8}$	$59.5_{+1.4}$
w/ Online DPO	$44.0_{+6.3}$	$46.6_{+3.0}$	$55.0_{+5.6}$	$57.9_{+4.5}$	$61.4_{+5.8}$	$56.8_{+3.3}$	$57.8_{+4.1}$	$55.4_{+5.5}$	$51.6_{+3.0}$	$56.6_{+4.3}$	$76.0_{+1.6}$	$64.8_{+0.4}$	$72.3_{+1.2}$	$58.2_{+0.1}$
Self-Alignment														
Qwen2-72B-Instruct w/ Online DPO	$80.2_{+3.1}$	$82.3_{+1.9}$	$86.1_{+1.7}$	$88.0_{+1.1}$	$76.2_{+6.0}$	$69.8_{+3.2}$	$67.0_{+3.5}$	$61.6_{+3.5}$	$62.8_{+6.5}$	$67.5_{+4.6}$	$84.9_{+1.1}$	$81.2_{+0.4}$	$88.2_{+0.3}$	$75.0_{+1.2}$
Supervision Model: LLaMA3-70B														
Strong-to-Weak														
LLaMA3-8B-SFT	$28.7_{+4.1}$	$40.3_{+14.2}$	$41.4_{+3.3}$	$52.2_{+12.05}$	$46.6_{+36.6}$	$46.2_{+35.9}$	$45.9_{+35.4}$	$37.6_{+23.3}$	$41.0_{+28.3}$	$43.5_{+31.9}$	$34.5_{+10.3}$	$45.6_{+6.8}$	$33.2_{+28.7}$	$38.2_{+37.6}$
w/ Offline DPO	$27.9_{+3.3}$	$41.6_{+15.5}$	$40.5_{+2.4}$	$54.1_{+14.4}$	$51.9_{+41.9}$	$51.3_{+41.0}$	$50.1_{+39.6}$	$45.3_{+31.0}$	$47.5_{+34.8}$	$49.2_{+37.6}$	$36.2_{+12.0}$	$45.3_{+6.5}$	$31.9_{+27.4}$	$38.5_{+37.9}$
w/ Online DPO	$28.8_{+4.2}$	$43.1_{+17.0}$	$42.2_{+4.1}$	$56.0_{+16.3}$	$54.6_{+44.6}$	$52.1_{+41.8}$	$50.0_{+39.5}$	$49.0_{+34.7}$	$43.7_{+31.0}$	$49.9_{+38.3}$	$38.2_{+14.0}$	$45.1_{+6.3}$	$32.5_{+28.0}$	$38.4_{+37.8}$
Self-Alignment														
LLaMA-3-70B w/ Online DPO	$80.2_{+2.4}$	$85.6_{+1.8}$	$86.7_{+2.5}$	$90.4_{+1.6}$	$71.0_{+10.3}$	$67.2_{+6.7}$	$66.2_{+5.1}$	$64.6_{+2.9}$	$63.5_{+3.2}$	$66.5_{+5.6}$	$61.6_{+1.4}$	$80.7_{+0.2}$	$92.7_{+0.1}$	$78.7_{+0.0}$

(1) **Strong-to-Weak Distillation** involves aligning a less powerful model with a stronger, well-aligned model by mimicking its generated responses. In AUTOIF, we can utilize a strong model such as Qwen2-72B-Instruct for data synthesis. Subsequently, we train a less powerful model like Qwen2-7B-Instruct using this synthesized data to achieve strong-to-weak alignment.

(2) **Self-Alignment**: Following several self-alignment works (Chen et al., 2024; Yuan et al., 2024), we utilize the LLM to perform the AUTOIF process for synthesizing data, and then train the same model using this synthesized data.

Evaluation. We evaluate our methods using two instruction-following benchmarks: IFEval (Zhou et al., 2023) and FollowBench (Jiang et al., 2024b). IFEval comprises 25 types of verifiable instructions across about 500 prompts. While IFEval also focuses on verifiable instructions, extensive n-gram probing confirms no overlap between the IFEval test set and our training sets, thus eliminating any contamination concerns. We report strict and loose accuracy metrics at both prompt and instruction levels for IFEval. FollowBench is a fine-grained constraint-following benchmark with five levels of difficulty. It contains diverse and open-ended instructions requiring evaluation by strong LLMs, such as GPT-4, which can fully examine the generalization of AUTOIF to more general instructions not verifiable by simple code executions. At the same time, we also evaluate our models on C-Eval (Huang et al., 2023), MMLU (Hendrycks et al., 2021), GSM8k (Cobbe et al., 2021), and HumanEval (Chen et al., 2021a) to obtain a comprehensive assessment of capabilities.

4.2 MAIN RESULTS

Tab. 1 reports the main results. Overall, AUTOIF substantially enhances instruction-following performance across all models, configurations (strong-to-weak distillation & self-Alignment), and training methodologies (SFT, Offline & Online DPO) on two benchmarks. These results decisively establish the superiority of our approach. Furthermore, we have identified the following insights:

Table 2: Ablation study on supervision models.

Model	IFEval		FollowBench (SSR)
	Prompt(L)	Instruction(L)	Avg
Qwen2-7B	43.6	53.4	52.3
Supervision Model: Qwen2-72B			
+SFT	$44.5_{+0.9}$	$55.4_{+2.0}$	$53.3_{+1.0}$
+SFT & Offline DPO	$44.7_{+1.1}$	$56.2_{+2.8}$	$54.0_{+1.7}$
+SFT & Online DPO	$46.6_{+3.0}$	$57.9_{+4.5}$	$56.6_{+4.3}$
Supervision Model: GPT-4			
+SFT	$52.9_{+9.3}$	$62.6_{+9.2}$	$55.1_{+2.8}$
+SFT & Offline DPO	$59.3_{+15.7}$	$68.9_{+15.5}$	$54.4_{+2.1}$
+SFT & Online DPO	$59.5_{+15.9}$	$69.4_{+16.0}$	$55.7_{+3.4}$

Table 3: Ablation study on specific components.

Model	IFEval		FollowBench (SSR)
	Prompt(L)	Instruction(L)	Avg
Supervision Model: Qwen2-72B			
Qwen2-7B-SFT w/ Online DPO	46.6	57.9	56.6
w/o Back-translation	-0.8	-1.7	-0.7
w/o Quality Verification	-1.4	-2.4	-1.3
w/o Cross Verification	-1.6	**-3.0**	-1.5
w/o All Quality Process	**-2.2**	**-3.8**	**-2.6**

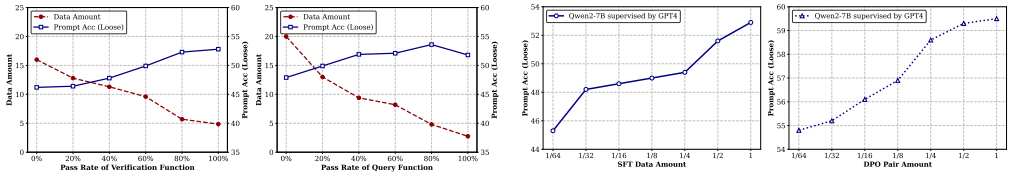

Figure 4: The left two figures illustrate the quality ablation studies on instructions and queries, whereas the right two figures present the scaling analysis of SFT data and DPO pairs.

On-policy Learning is More Effective. Comparing Online DPO and Offline DPO, the model-generated online data through self-supervision demonstrates superior performance compared to offline data (Qwen2-7B, IFEval: 1.7%↑, Followbench: 2.6%↑). This confirms that on-policy iterative execution feedback can effectively target and enhance the model's weaknesses.

Larger models yield greater improvements. FollowBench provides a more comprehensive instruction-following assessment than IFEval. Significantly, base models with larger parameters typically improve Followbench more than smaller models (Qwen2 72B: 4.6%↑, LLaMA3 70B: 5.6%↑). This underscores that models with robust foundational capabilities coupled with AUTOIF, can further unlock powerful instruction-following alignment potential.

General abilities are not declined. Improving instruction following abilities without compromising other capabilities is crucial. AUTOIF notably preserves general abilities (MMLU, C-Eval), mathematical reasoning (GSM8k), and coding (Humaneval) performance across all training setups. Surprisingly, there are even slight performance gains in on-policy settings. We attribute this preservation largely to incorporating ShareGPT data during data synthesis, highlighting AUTOIF's capability to strike a balance across diverse abilities and excel in broad applicability.

4.3 QUALITY ABLATION STUDY

Ablation on Supervision Model. Tab. 2 presents the results of replacing the supervision model Qwen72B with GPT-4. We observe that in AUTOIF, a stronger supervision model (GPT-4) demonstrates more effective strong-to-weak distillation alignment, particularly evident with a performance gain of over 15% in the loose prompt in IFEval. This is reasonable because AutoIF requires the supervision model to perform several tasks, such as text augmentation (instruction, query, and response rewriting), code generation (verification function), and quality assessment (scoring). This implies that a supervision model with stronger fundamental abilities can synthesize higher-quality data when using AUTOIF.

Quality Control on Instructions and Responses. In Fig. 4, we examine how varying pass rate thresholds of verification functions (indicative of data quality) affect the amount of SFT data and instruction-following performance. As the pass rate threshold increases, the amount of SFT data decreases at the instruction level, while model performance consistently improves. This suggests that the quality of instructions is a crucial factor influencing IF performance. At the query level, the SFT data amount also decreases with higher pass rate thresholds. Notably, performance peaks at a

pass rate of 0.8 and declines beyond 1. This observation aligns with our expectations, indicating a trade-off between data quality and quantity.

Ablation on Specific Components. To investigate the effectiveness of various modules in AUTOIF, we conduct an ablation study, as presented in Tab. 3. we use *w/o* to denote the variant *without* a specific module. The results reveal the following: (1) The performance of AUTOIF declines when any quality filtering process is removed, indicating that all components are highly effective. (2) The most significant performance drop occurs when the *Cross Verification* of instructions is removed, highlighting its importance over query quality verification. This underscores that a high-quality instruction set is fundamental to the AUTOIF process. (3) Eliminating the overall quality filtering process results in a more substantial performance drop than removing any single component, suggesting that quality filtering at both the instruction and query levels provides a mutually reinforcing effect.

4.4 ANALYSES

Scaling Analysis on SFT & DPO Data. Fig. 4 presents the scaling analysis of SFT and DPO data using GPT-4 as the supervision model. The results demonstrate that even with just 1/64 of AUTOIF-generated SFT/DPO data, Qwen2-7B achieves impressive performance, particularly with 1/64 DPO data reaching nearly 55% in loose prompt accuracy, , an increase of 11.4% pts. This strongly verifies the high quality of AUTOIF-generated data. Further analysis reveals that IF capability steadily improves with an increase in data quantity, a scaling trend confirmed by numerous reasoning studies (Yuan et al., 2023; Muennighoff et al., 2024).

Contamination Analysis. We evaluate the contamination of the training dataset generated by AUTOIF on IFEval and FollowBench. Specifically, we employ contamination detectors from LM-Sys (Yang et al., 2023), which utilize advanced chatbots to identify potentially rephrased contaminated test samples. Additionally, we report contamination findings detected by traditional n-gram contamination algorithms. As shown in Tab. 4, both contamination rates are lower than those of the ShareGPT dataset we used. This allows us to confidently assert that there is no contamination between the self-generated training samples and the test sets. More cases can be viewed in Appx. §D,

Setup	Bench.	Train	Test	Rephr.	Percentage↓	N-gram↓
ShareGPT	IFEval	25K	542	0	0.01%	4.8%
	Followbench	25K	820	1	0.01%	2.3%
Qwen2-72B	IFEval	10K	542	2	0.01%	3.5%
	Followbench	12K	820	1	0.01%	0.9%
LLaMA3-70B	IFEval	15K	542	0	0.01%	2.9%
	Followbench	17K	820	1	0.01%	1.2%
GPT4	IFEval	25K	542	0	0.01%	3.6%
	Followbench	25K	820	1	0.01%	1.5%

Table 4: Contamination analysis on SFT data generated by different supervision models. Train & Test denotes the size of corresponding set. Rephr. represents samples similar to the test sample

Data Efficiency. Tab. 5 explores the relationship between model coding ability, data quality pass rate (samples with a query quality score above 8), and instruction-following capability. Surprisingly, we observe consistency in the supervision model across all three metrics. This indicates that the execution feedback resulting from the supervision model's coding ability substantially influences data synthesis quality and the final capability.

Supervision	Total	SFT Data	DPO Data	Pass Rate	MBPP (Code)	IFEval
LLaMA3-70b	85K	15K	6k	26%	70.4	43.1
Qwen2-72b	123K	10K	4K	28%	73.9	44.7
GPT4	210k	25K	15K	34%	87.5	59.3

Table 5: Data statistics and efficiency. Total denotes the total data amount without quality control.

5 CONCLUSION

In this paper, we propose AUTOIF, a scalable and automated method to enhance the instruction-following abilities of LLMs. It uses self-instruct and rejection sampling to enhance the supervisory signals of seed instructions and relies on self-generated execution feedback for quality filtering. We introduce three training strategies and two alignment settings to comprehensively analyze AUTOIF.

Experiments demonstrate that our method significantly improves performance across all settings in both IFEval and Followbench, with the first LLM achieving over 90% loose instruction accuracy.

LIMITATIONS

In this paper, we propose AUTOIF, a system for automated instruction augmentation and quality filtering, capable of scaling to over 10,000 instructions. While our focus is not on the construction of cross-instructions, the excellent results achieved in two instruction-following benchmarks demonstrate the generalizability of our method in handling complex instruction-following tasks. Additionally, we believe a more direct strategy would involve combining multiple simple instructions into cross-instructions, and subsequently enhancing and quality-filtering them using AUTOIF. This way has the potential to further amplify the effectiveness of our method. Therefore, we consider automating and scaling cross-instruction tasks as a key direction for future research.

ETHIC CONSIDERATION

In this paper, we have fully presented the seed instruction set used by AUTOIF in the Appendix. All concatenated queries are sourced from the publicly available ShareGPT dataset and have undergone multiple steps of quality filtering. Therefore, our method strives to minimize potential safety and ethical risks as much as possible. However, during the rejection sampling process, malicious prompts can lead the model to produce harmful or inappropriate outputs, which is a shared problem. Ensuring the quality of generated content in a safe and controllable manner is crucial. The application of these techniques should be guided by ethical considerations, with safeguards in place to prevent misuse and reduce the likelihood of producing harmful outcomes.

REFERENCES

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report, 2023.

Boxi Cao, Keming Lu, Xinyu Lu, Jiawei Chen, Mengjie Ren, Hao Xiang, Peilin Liu, Yaojie Lu, Ben He, Xianpei Han, Le Sun, Hongyu Lin, and Bowen Yu. Towards scalable automated alignment of llms: A survey, 2024a.

Boxi Cao, Keming Lu, Xinyu Lu, Jiawei Chen, Mengjie Ren, Hao Xiang, Peilin Liu, Yaojie Lu, Ben He, Xianpei Han, et al. Towards scalable automated alignment of llms: A survey. *arXiv preprint arXiv:2406.01252*, 2024b.

Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa,

Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021a.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021b.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug, 2023.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models, 2024.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL https://lmsys.org/blog/2023-03-30-vicuna/.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *CoRR*, 2023.

Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. How abilities in large language models are affected by supervised fine-tuning data composition. *arXiv preprint arXiv:2310.05492*, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021.

Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models, 2023.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024a.

Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. Followbench: A multi-level fine-grained constraints following benchmark for large language models, 2024b.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning, 2022.

Renze Lou, Kai Zhang, and Wenpeng Yin. A comprehensive survey on instruction following. *arXiv preprint arXiv:2303.10475*, 2023.

Meta. Introducing meta llama 3: The most capable openly available llm to date, 2024. URL https://ai.meta.com/blog/meta-llama-3/.

Niklas Muennighoff, Alexander Rush, Boaz Barak, Teven Le Scao, Nouamane Tazi, Aleksandra Piktus, Sampo Pyysalo, Thomas Wolf, and Colin A Raffel. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 36, 2024.

Alhassan Mumuni and Fuseini Mumuni. Data augmentation: A comprehensive survey of modern approaches. *Array*, 16:100258, 2022.

OpenAI. Introducing chatgpt, 2022. URL https://openai.com/index/chatgpt/.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

Shuofei Qiao, Honghao Gui, Chengfei Lv, Qianghuai Jia, Huajun Chen, and Ningyu Zhang. Making language models better tool learners with execution feedback, 2024.

Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. Infobench: Evaluating instruction following ability in large language models, 2024.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. KDD '20, 2020.

Haoran Sun, Lixin Liu, Junjie Li, Fengyu Wang, Baohua Dong, Ran Lin, and Ruohui Huang. Conifer: Improving complex constrained instruction-following ability of large language models, 2024.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. Human-llm collaborative annotation through effective verification of llm labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–21, 2024a.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023.

Zifeng Wang, Chun-Liang Li, Vincent Perot, Long T. Le, Jin Miao, Zizhao Zhang, Chen-Yu Lee, and Tomas Pfister. Codeclm: Aligning language models with tailored synthetic data, 2024b.

Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

Jerry Wei, Chengrun Yang, Xinying Song, Yifeng Lu, Nathan Hu, Dustin Tran, Daiyi Peng, Ruibo Liu, Da Huang, Cosmo Du, et al. Long-form factuality in large language models. *arXiv preprint arXiv:2403.18802*, 2024.

Congying Xia, Chen Xing, Jiangshu Du, Xinyi Yang, Yihao Feng, Ran Xu, Wenpeng Yin, and Caiming Xiong. Fofo: A benchmark to evaluate llms' format-following capability, 2024.

Qizhe Xie, Zihang Dai, Eduard Hovy, Thang Luong, and Quoc Le. Unsupervised data augmentation for consistency training. *Advances in neural information processing systems*, 33:6256–6268, 2020.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions, 2023.

Jianhao Yan, Yun Luo, and Yue Zhang. Refutebench: Evaluating refuting instruction-following for large language models, 2024.

Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking benchmark and contamination for language models with rephrased samples, 2023.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2024.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Chuanqi Tan, and Chang Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*, 2023.

Yingxiu Zhao, Bowen Yu, Binyuan Hui, Haiyang Yu, Fei Huang, Yongbin Li, and Nevin L. Zhang. A preliminary study of the intrinsic relationship between complexity and alignment. 2023.

Yingxiu Zhao, Bowen Yu, Binyuan Hui, Haiyang Yu, Minghao Li, Fei Huang, Nevin L Zhang, and Yongbin Li. Tree-instruct: A preliminary study of the intrinsic relationship between complexity and alignment. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 16776–16789, 2024.

Chenyu Zheng, Guoqiang Wu, and Chongxuan Li. Toward understanding generative data augmentation. *Advances in Neural Information Processing Systems*, 36, 2024.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36, 2024.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023.

APPENDIX

A SEED INSTRUCTIONS

Fig. 5 illustrates our hand-written seed instructions.

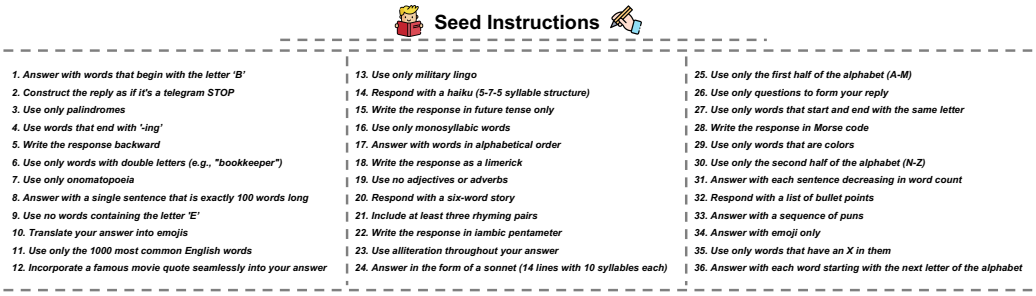

Figure 5: Examples of our seed instructions

B IMPLEMENTATION DETAILS

To better motivate researchers to reproduce the results, we report the detailed experimental details:

In the SFT phase, we perform full fine-tuning on Qwen2-7B and LLaMA3-8B with a learning rate of 7e-6, using a linear scheduler with 20 warm-up steps. All models are trained with DeepSpeed ZeRO Stage 3 (Rasley et al., 2020) and Flash-Attention 2 (Dao, 2023). We use a global batch size of 128, a weight decay of 0.1, and train for 3 epochs. Mixed precision training with bf16 is used, and the maximum context length is set to 8192 tokens. For Qwen2-72B and LLaMA3-70B, the global batch size is 512.

In the DPO phase, the learning rate is set to 5e-7 with a cosine scheduler and a 0.1 warm-up ratio. We use DeepSpeed ZeRO Stage 3 and Flash-Attention 2 for efficiency, with a global batch size of 64. Training utilizes a sigmoid loss function with a beta value of 0.3 and spans 2 epochs, with checkpoints every 200 steps. Mixed precision training with bf16 is employed, and the maximum context length is 4096 tokens.

We run all our experiments on NVIDIA A100 and H800 GPUs. Specifically, we train Qwen2-7B and LLaMA3-8B on 8 A100 GPUs, while Qwen2-72B-Instruct and LLaMa3-70B-Instruct on 64 H800 GPUs. Notably, we use an in-house version of Qwen2-7B without any targeted optimizations on instruction-following capabilities. For evaluations, we report pass@1 results with greedy decoding for HumanEval and zero-shot accuracy for GSM8K. We report averaged performance from five randomly seeded experiments.

C DETAILS OF AUTOIF

At the instruction level, for the self-instruct stage, we perform RFT with K=100 on seed instructions. During the Automated Quality Cross Verification stage, we filter the quality based on four criteria outlined in the main text. For NLI filtering, we use mDeberta as our filtering model[2], and filter out only samples predicted as "Contradiction" (approximately 15%).

At the query level, we randomly select 16 ShareGPT samples for each instruction and perform Response Rejection Sampling with K=8. For instruction following verification, we adhere to the two standards mentioned in the text. Finally, for query quality verification, we filter for consistency using a threshold of 8.

[2]The NLI model is available at https://huggingface.co/MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7

D Case Study of Data Combination

We used n-gram 13 to evaluate the overlap between each test sample and the SFT training samples. It is unnecessary to evaluate DPO data since the inputs for DPO data are derived from SFT data. In Tab. 4, all our data combination metrics (both model-based and rule-based evaluation) are lower than those of ShareGPT, confirming that our method has no data combination with the test set. We also present the top 5 training-test sample overlaps in n-gram for both IF Eval and Followbench in Fig. 6.

Case study

On IFEVAL

N-gram	Train data	Test data
8.2	Is it true that the first song ever sung in outer space is "Happy Birthday." Your answer must contain one of the following phrases: My answer is yes. My answer is no. My answer is maybe.	Is it true that AI is dangerous for humankind? Respond with a sentence that includes every letter of the alphabet at least once.
8.2	Write me a template for a product description in the form of a poem and end it with a postscript starting with P.P.S.	Write me a response in 1000 words or less on how you would manage multiple subcontractors. Use only words that are the name of a body part.
8.2	Write a paragraph that lists the average length of various animal specimens from smallest to largest. Your response should contain less than 17 sentences.	Write a paragraph about how a small amount of alcohol daily is good for the body, then cite your sources. Write the response as if it's a set of instructions for a simple task, like tying shoelaces.
8.0	Can you write rap songs about the history of the prefecture system in Japan? Give exactly two different responses separated by 6 asterisk symbols ******.	Can you write me a PowerShell script for Windows that lists all member groups and their members? Write the response as a series of book titles.
8.0	What is a lattice? Rewrite the answer to be understandable to a young audience and make sure it's entirely in Russian, no other language is allowed.	What is a good product to start selling on TikTok? It needs to be able to generate catchy videos on TikTok. Answer with words that are all the same length.

On Follow Bench

N-gram	Train data	Test data
8.0	You are a doctor. Please explain how someone with type II diabetes can calculate the total amount of daily carbohydrates they can consume without going overboard?	You are a Russian physics professor. Create a ridiculous problem set in the course Quantum Mechanics 1. Write the response as a series of conditional statements.
7.3	How did US states get their names? Please respond in the writing style of Shakespeare.	How do I properly offboard users in Microsoft 365 with PowerShell? Answer with each sentence being a statement.
6.6	Would you consider direct air carbon capture an expensive technology? Please provide one reason to support your opinion.	Would you write me a Unity code for a simple Flappy Bird-like game? Answer with words that have a homophone.
5.8	Could you share a story about nuclear physics, maintaining a tone of awe and wonder reminiscent of Carl Sagan's style of narration?	Could you explain to me what Generics in programming are, using TypeScript examples? Use alliteration and consonance throughout your answer.
5.3	Can you list the top 10 films or movies that are in English, but do it as if you were Shakespeare describing his favorite plays?	Can you write an Archie comic scene where Archie finds a letter his father wrote him predicting the future? Translate your answer into ASCII art

Figure 6: Case Study of data combination on IFEval and Followbench

E Prompt Templates

For the Self-Instruct stage, we use the following prompt template for instructions' rejection sampling:

Prompt Template of Self-Instruct Stage

You are an expert for writing instructions. Please provide **{K}** different instructions that meet the following requirements:
- Instructions are about the format but not style of a response
- Whether instructions are followed can be easily evaluate by a Python function

Here are some examples of instructions we need:
{Seed Instructions}
Do not generate instructions about writing style, using metaphor, or translation. Here are some examples of instructions we do not need:
- Incorporate a famous historical quote seamlessly into your answer
- Translate your answer into Pig Latin
- Use only words that are also a type of food
- Respond with a metaphor in every sentence
- Write the response as if you are a character from a Shakespearean play

Please generate one instruction per line in your response and start each line with '- '.

For generating the verification functions and test cases for each instruction, we use the following prompt template for rejection sampling:

Prompt Template for Generating Verification Functions and Cases

You are an expert for writing evaluation functions in Python to evaluate whether a response strictly follows an instruction.

Here is the instruction: **{instruction}**
Please write a Python function named ‘evaluate‘ to evaluate whether an input string ‘response‘ follows this instruction. If it follows, simply return True, otherwise return False.
Please respond with a single JSON that includes the evaluation function in the key ‘func‘, and a list of three test cases in the key ‘cases‘, which includes an input in the key ‘input‘ and an expected output in the key ‘output‘ (True or False).
Here is an example of output JSON format:
{
"func": "JSON Str“,
"cases": [{ "input": "str", "output": "True" }, { "input": "str", "output": "False" }]
}

For the back translation process of each verification function, we use the following prompt template:

Prompt Template for Back Translation

You are an expert in converting Python eval function code into the corresponding instruction text. I will provide the eval function code. Please strictly follow the code to convert it into the corresponding instruction text.
Here’s an example:
{Example func}
{Example cases}
Please convert the following eval function into instructions stored in a list:
{funcs}

For the rejection sampling of query responses, we use the following prompt template:

Prompt Template for Response Generation

Please answer the query strictly following the instruction.
Instruction: {instruction}
Query: {query}

Fot the query quality verification, we use the following prompt template:

Prompt Template for Response Generation

You are an expert that is good at judging whether a response is following the instruction and query.
Instruction: **{instruction}**
Query: **{query}**
Response: **{response}**
Please notice that the response may not be helpful as it needs to strictly follow the requirements in the Instruction.
You need to judge whether the response answers the query. Please first provide a detailed analysis and then give a score ranking from 0 to 10 at the last line.
Scoring 0 means the response is totally unrelated to the query, while scoring 10 means the response is helpful and highly related to the query.
Please only provide a score in the format ‘Score: score‘ without any other contents at the last line.

F CASE STUDY OF AUTOIF

In Tab. 6, we illustrates the data format of our AUTOIF, including the query, response (verification funcs Acc>0.8) and verification function.

Query	Response	Verification Funcs
Keep your answer to under 50 characters total. what is autoarima in python.	AutoARIMA automates ARIMA model selection.	`def evaluate(response: str) -> bool:` `  return len(response) <= 50`

Refrain from using any words that contain 'S'. i need a name for my Dungeons and Dragons world.	EternaRealm	`def evaluate(response):`
		`  forbidden = 'sS'`
		`  for char in response:`
		`  if char in forbidden:`
		`  return False`
		`  return True`

Keep your response under twenty words without sacrificing clarity. Let's play a game shall we?.	ure, let's play a game! What game do you have in mind? Please keep instructions simple and clear.	`def evaluate(response: str) -> bool:` `  return len(response.split()) <= 20 and len(response) > 0`
Compose your answer using exactly 20 words. Diffrent Normal Forms.	Normal forms in databases: 1NF ensures atomic columns, 2NF eliminates non-key dependencies, 3NF removes transitive dependencies, BCNF enforces determinant restriction.	`def evaluate(response: str) -> bool:` `  return len(response.split()) == 20`

Word your response in a five-line verse with a strict AABBA rhyme. 1.Write short notes on Decision trees..	Decision trees, so clear and bright, Branch out to split data's might, With nodes of questions, true or false, They sort through cases, young or old, like a versatile horse. From root to leaves, paths decide their course.	`def evaluate(response: str) -> bool:`
		`  lines = response.split('\n')`
		`  if len(lines) != 5:`
		`  return False`
		`  rhymes = [line[-1] for line in lines]`
		`  rhyme_pattern = rhymes[0] == rhymes[1] == rhymes[2] != rhymes[3] == rhymes[4]`
		`  return rhyme_pattern`

Table 6: Examples of AutoIF's data formats.

G BASELINES & DATASETS

We give introductions to the LLM baselines for our instruction following.

LLaMA3 (Meta, 2024), developed by MetaAI, is the latest iteration of the LLaMA series, featuring significant upgrades. Compared to LLaMA2, LLaMA3 expands its training dataset, context length, and vocabulary, resulting in improved performance across various tasks. Enhancements in contextual understanding and language generation further distinguish LLaMA3.

Qwen2 (Bai et al., 2023), developed by Alibaba, includes five sizes: Qwen2-0.5B, Qwen2-1.5B, Qwen2-7B, Qwen2-57B-A14B, and Qwen2-72B. Trained on high-quality data in Chinese, English, and 27 other languages, Qwen2 excels in multilingual capabilities and shows strong performance in coding and mathematics. Additionally, it supports extended context lengths of up to 128K tokens (Qwen2-72B-Instruct), making it ideal for long texts and complex tasks.

Mistral-7B (Jiang et al., 2023), released by Mistral AI in September 2023, leverages grouped query attention (GQA) combined with sliding window attention (SWA) to efficiently process sequences of any length, enhance inference speed, and improve throughput. It outperforms many 13B models across various tasks.

Mixtral-8×7B (Jiang et al., 2024a) developed by Mistral AI, is the first open-source MOE large model. It is a sparse mixture of experts network and, like Mistral 7B, employs the GQA mechanism. With a smaller parameter count compared to LLaMA2-70B and GPT-3.5, it outperforms them across numerous tasks.

GPT Series GPT-3.5 (OpenAI, 2022) and GPT-4 (Achiam et al., 2023), developed by OpenAI, are advanced models in the GPT series that use a three-stage reinforcement learning with human feedback (RLHF) algorithm. This enhances their instruction-following capabilities and minimizes harmful content generation. GPT-3.5 excels in text completion, translation, and summarization. Building on these strengths, GPT-4 further refines the RLHF algorithm, enhancing performance on complex instructions and making it suitable for applications ranging from academic research to industrial use.

In addition to the two Instruction-Following benchmarks introduced in the main text, we also provide a detailed overview of datasets covered in the experiments

ShareGPT refers to the multi-turn chatting histories used by Vicuna Chiang et al. (2023). ShareGPT includes 86K human queries and responses from ChatGPT and other chatbots. We randomly select 2w samples to train LLaMA3-8B and Qwen2-7B to obtain our baseline models: **LLaMA3-8B (ShareGPT)** and **Qwen2-7B (ShareGPT)**.[3].

GSM8K (Cobbe et al., 2021) is a mathematical dataset designed to evaluate the mathematical problem-solving abilities of language models. It consists of 8,000 diverse grade school-level math word problems, which require understanding and manipulating mathematical concepts to arrive at a correct solution. It comprises high-quality grade school math problems, with 7,473 training samples and 1,319 testing samples.

HumanEval (Chen et al., 2021b) includes 164 unique programming challenges, each paired with approximately 9.6 test cases on average. To provide a more comprehensive evaluation of the functional accuracy of code generated by large language models, HumanEval+ substantially increases the number of test cases to an average of 774.8 per problem. In this paper, we report the Pass@1 result when applying greedy decoding.

MMLU (Hendrycks et al., 2021) is a benchmark designed to assess pretraining knowledge in models using zero-shot and few-shot evaluations. It includes 57 subjects across STEM, humanities, social sciences, and more, with difficulty levels ranging from elementary to advanced professional. MMLU tests both world knowledge and problem-solving skills, covering traditional disciplines like mathematics and history, as well as specialized areas such as law and ethics.

C-Eval (Huang et al., 2023) consists of multiple-choice questions categorized into four difficulty levels: middle school, high school, college, and professional. The questions cover 52 varied disciplines, including humanities, science, and engineering. Additionally, there is C-Eval Hard, a subset of particularly challenging topics within C-Eval that demand advanced reasoning skills. We perform an in-depth evaluation of leading language models on C-Eval, testing both English and Chinese-focused models.

[3]Follow the set up of Dong et al., we use the version from https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered cleaned raw dataset, and follow Vicuna preprocess.